Optimization Methods for Inverse Kinematics of a Planar Arm

Paweekorn Buasakorn
School of Automation Science and Technology, Xi'an Jiaotong University
May 10, 2026

Source code: github.com/B-Paweekorn/invkin-cvx-optimization

Abstract

Inverse kinematics (IK) is a problem to computing joint angles that place a robot's end-effector at a desired goal is can classically solved by geometric or algebraic closed-form methods. These work well for low-dimensional arms but become intractable as the number of joints grows: deriving closed-form solutions for $n > 3$ joints is analytically difficult, and enumerating configuration space by brute-force grid search requires $O(k^n)$ storage for a k-point grid per joint exponential in dimension. Optimization-based IK avoids both problems: minimising the squared end-effector error $f(\mathbf{q}) = \frac{1}{2}\|\mathrm{FK}(\mathbf{q}) - \mathbf{g}\|^2$ requires only $O(n^2)$ storage for gradient and Hessian, and scales gracefully with dimensionality. This report implements and analyses eight such methods five unconstrained (Gradient Descent, Steepest Descent, Newton/GN, BFGS, Conjugate Gradient) and three constrained (Newton+KKT, Infeasible Newton, Interior-Point Log-Barrier) on a planar n-joint arm environment adapted from PythonRobotics [1]. We compare convergence speed, memory cost, and robustness under joint limits, equality constraints, and obstacle avoidance, and provide practical guidance on which method to choose for each constraint type.

Introduction

Classical IK approaches and their limits. Geometric and algebraic methods solve IK by exploiting the specific kinematic structure of a manipulator to derive closed-form joint angles [2, 3]. For a 2-DOF planar arm the solution is a simple trigonometric formula; for a 3-DOF arm with a spherical wrist it extends to an analytic decoupling. Beyond three or four joints, however, deriving the closed form becomes increasingly complex each additional degree of freedom introduces new coupling terms, and for redundant arms ($n > m$) the solution is non-unique, making a single analytic formula impractical to construct and use. Geometric methods also struggle when joint limits, obstacles, or secondary objectives (e.g. posture preference, singularity avoidance) must be incorporated simultaneously.

Numerical search and the curse of dimensionality. A natural alternative is to search the n-dimensional configuration space numerically. Uniform grid search with k samples per joint evaluates k^n points exponential in n and entirely impractical beyond 5–6 joints. Even randomised samplers such as RRT or PRM [4] scale poorly with dimension for precision-critical IK tasks.

Optimization as a scalable solution. Casting IK as minimising $f(\mathbf{q}) = \frac{1}{2}\|\mathrm{FK}(\mathbf{q}) - \mathbf{g}\|^2$ requires only $O(n)$–$O(n^2)$ memory and handles constraints (joint limits, obstacles) without exponential blowup. Note that f is *nonconvex* — FK involves trigonometric functions of cumulative joint angles, producing multiple local minima (e.g. elbow-up vs. elbow-down). All methods here are therefore *local* optimisers that find *a* valid configuration, not necessarily the global minimum, making IK an ideal test case for nonconvex optimisation in practice.

This work. We systematically implement and compare eight optimisation algorithms for IK on a planar n-joint arm, examining how each trades off convergence rate, memory, and constraint-handling capability. The environment is adapted from the open-source PythonRobotics n-joint arm-to-point controller [1]. All source code is available at github.com/B-Paweekorn/invkin-cvx-optimization.

Problem Formulation

Kinematics. Consider a planar arm with n revolute joints, link lengths $\ell_1, \dots, \ell_n$, and joint configuration $\mathbf{q} = (q_1, \dots, q_n)^\top \in \mathbb{R}^n$, $q_i \in [-\pi, \pi]$. The *forward kinematics* (FK) map is

$$\mathrm{FK}(\mathbf{q}) = \sum_{i=1}^{n} \ell_i \begin{pmatrix} \cos\left(\sum_{j=1}^{i} q_j\right) \\ \sin\left(\sum_{j=1}^{i} q_j\right) \end{pmatrix} \in \mathbb{R}^2. \tag{1}$$

Inverse kinematics (IK) seeks $\mathbf{q}$ such that $\mathrm{FK}(\mathbf{q}) = \mathbf{g}$ for a given target $\mathbf{g} \in \mathbb{R}^2$.

Optimization objective. For a redundant planar arm ($n > 2$, task space dimension $m = 2$) we minimize the squared end-effector error (experiments use $n = 10$, $\ell_i = 2$):

$$\min_{\mathbf{q} \in \mathbb{R}^n} \; f(\mathbf{q}) = \tfrac{1}{2} \|\mathrm{FK}(\mathbf{q}) - \mathbf{g}\|^2. \tag{2}$$

Because FK involves trigonometric functions of cumulative joint angles, f is *nonconvex* with multiple local minima (distinct arm postures reaching the same goal). We apply convex optimization methods *locally*: at each iterate $\mathbf{q}_k$, the Gauss-Newton model $\mathbf{H}_k \approx \mathbf{J}_k^\top \mathbf{J}_k + \lambda \mathbf{I} \succ 0$ forms a convex quadratic approximation of f, whose minimiser gives a descent direction. Iterating converges to a local minimum — sufficient for IK, where any $\|\mathrm{FK}(\mathbf{q}) - \mathbf{g}\| < \varepsilon$ is a valid solution.

Writing $\mathbf{e}(\mathbf{q}) = \mathrm{FK}(\mathbf{q}) - \mathbf{g}$ and letting $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{2 \times n}$ be the geometric Jacobian, the gradient and (Gauss–Newton) Hessian are

$$\nabla f(\mathbf{q}) = \mathbf{J}(\mathbf{q})^\top \mathbf{e}(\mathbf{q}), \qquad \mathbf{H}(\mathbf{q}) \approx \mathbf{J}(\mathbf{q})^\top \mathbf{J}(\mathbf{q}) + \lambda \mathbf{I} \succ 0, \tag{3}$$

where the Gauss–Newton approximation drops the small second-order FK correction (valid near a solution) and $\lambda > 0$ regularises near singularities. Differentiating (1) with respect to q_j gives the j-th column of $\mathbf{J}$:

$$\mathbf{J}_{:,j}(\mathbf{q}) = \sum_{k=j}^{n} \ell_k \begin{pmatrix} -\sin\left(\sum_{i=1}^{k} q_i\right) \\ \cos\left(\sum_{i=1}^{k} q_i\right) \end{pmatrix}, \quad j = 1, \dots, n. \tag{4}$$

Joint j rotates every link from j to n, so its column sums their velocity contributions. Assembling all n columns yields the full $2 \times n$ matrix:

$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} -\sum_{k=1}^{n} \ell_k \sin\left(\sum_{i=1}^{k} q_i\right) & -\sum_{k=2}^{n} \ell_k \sin\left(\sum_{i=1}^{k} q_i\right) & \cdots & -\ell_n \sin\left(\sum_{i=1}^{n} q_i\right) \\ \sum_{k=1}^{n} \ell_k \cos\left(\sum_{i=1}^{k} q_i\right) & \sum_{k=2}^{n} \ell_k \cos\left(\sum_{i=1}^{k} q_i\right) & \cdots & \ell_n \cos\left(\sum_{i=1}^{n} q_i\right) \end{bmatrix} \in \mathbb{R}^{2 \times n}. \tag{5}$$

Common iteration. Every method below produces iterates $\mathbf{q}_{k+1} = \mathbf{q}_k + \alpha_k \mathbf{d}_k$, differing only in how the *search direction* $\mathbf{d}_k$ and *step size* α_k are chosen. The following diagram shows the logical progression, each method a direct response to the previous one's limitation.

1. Gradient Descent —(adaptive α)→ 2. Steepest Descent —(use $\mathbf{H}$)→ 3. Newton (Gauss–Newton) —(approx. $\mathbf{H}^{-1}$)→ 4. BFGS —($O(n)$ mem.)→ 5. Conj. Grad.

1 Algorithm 1 — Gradient Descent (Fixed Step Size)

The simplest descent strategy: move opposite the gradient by a fixed scalar α,

$$\mathbf{q}_{k+1} = \mathbf{q}_k - \alpha \nabla f(\mathbf{q}_k) = \mathbf{q}_k - \alpha \mathbf{J}_k^\top \mathbf{e}_k. \tag{6}$$

The update $-\mathbf{J}^\top \mathbf{e}$ "pulls" joints to reduce the current end-effector error; no matrix inversion is needed. For f with L-Lipschitz gradient, stability requires $\alpha < 2/L$, where $L = \sigma_{\max}^2(\mathbf{J})$. In practice $\alpha = 0.0005$ is a hand-tuned constant.

Theorem 1.1. *For convex L-smooth f, gradient descent with $\alpha = 1/L$ gives $f(\mathbf{q}_k) - f^* \le L\|\mathbf{q}_0 - \mathbf{q}^*\|^2/(2k)$:* linear *$(O(1/k))$ convergence. IK is nonconvex, so this bound does not apply; in practice GD converges to a local minimum.*

At kinematic singularities $\nabla f = \mathbf{J}^\top \mathbf{e}$ may vanish while $\mathbf{e} \neq \mathbf{0}$; a small random kick to $\mathbf{q}$ escapes such traps.

Algorithm 1 Gradient Descent IK

Input: $\mathbf{q}_0$, target $\mathbf{g}$, fixed step α, tolerance ε
Output: IK solution $\mathbf{q}$

$\mathbf{q} \leftarrow \mathbf{q}_0$
repeat
  $\mathbf{e} \leftarrow \mathrm{FK}(\mathbf{q}) - \mathbf{g}$; **if** $\|\mathbf{e}\| < \varepsilon$ **break**
  $\mathbf{g}_\nabla \leftarrow \mathbf{J}(\mathbf{q})^\top \mathbf{e}$
  if $\|\mathbf{g}_\nabla\| < 10^{-8}$ **then** $\mathbf{q} \leftarrow \mathbf{q}$ + random kick ▷ escape singularity
  else $\mathbf{q} \leftarrow \mathbf{q} - \alpha\, \mathbf{g}_\nabla$
  end if
until max iterations

2 Algorithm 2 — Steepest Descent with Backtracking Line Search

GD uses the same α regardless of local curvature: too small on flat regions, too large in narrow valleys (causing zigzag). Steepest descent keeps the gradient direction but selects α_k automatically via the *Armijo condition*:

$$f(\mathbf{q} - \alpha\, \mathbf{g}_\nabla) \le f(\mathbf{q}) - c_1 \alpha \|\mathbf{g}_\nabla\|^2, \quad c_1 = 10^{-4}. \tag{7}$$

Definition 2.1 (Armijo Backtracking)**.** Starting from $\alpha = 1$, set $\alpha \leftarrow \rho\alpha$ (with $\rho = 0.5$) until (7) holds. Formally,

$$\alpha^* = \max\{\rho^j :\ f(\mathbf{q} + \rho^j \mathbf{d}) \le f(\mathbf{q}) + c_1 \rho^j \mathbf{g}_\nabla^\top \mathbf{d},\ j = 0, 1, \ldots\}.$$

The step is large where f is flat and small where f curves sharply—automatically. The convergence class remains linear (Theorem 1.1), but the adaptive step drastically cuts iteration counts over a poorly tuned fixed α. Ill-conditioning of $\mathbf{J}^\top \mathbf{J}$ still causes zigzag near the solution; improving the *direction* is the key insight of Algorithm 3.

Algorithm 2 Steepest Descent IK (Armijo backtracking)

Input: $\mathbf{q}_0$, $\mathbf{g}$, $c_1 = 10^{-4}$, $\rho = 0.5$, ε
Output: IK solution $\mathbf{q}$

$\mathbf{q} \leftarrow \mathbf{q}_0$
repeat
  $\mathbf{e} \leftarrow \mathrm{FK}(\mathbf{q}) - \mathbf{g}$; **if** $\|\mathbf{e}\| < \varepsilon$ **break**
  $\mathbf{g}_\nabla \leftarrow \mathbf{J}(\mathbf{q})^\top \mathbf{e}$; $\mathbf{d} \leftarrow -\mathbf{g}_\nabla$
  $\alpha \leftarrow \textsc{Backtrack}(f, \mathbf{q}, \mathbf{d}, \mathbf{g}_\nabla)$
  $\mathbf{q} \leftarrow \mathbf{q} + \alpha\, \mathbf{d}$
until max iterations

3 Algorithm 3 — Newton's Method (Gauss–Newton)

Newton's method replaces the gradient direction with the minimiser of the local quadratic:

$$\min_{\mathbf{d}} \; f(\mathbf{q}) + \nabla f(\mathbf{q})^\top \mathbf{d} + \tfrac{1}{2}\mathbf{d}^\top \mathbf{H}(\mathbf{q})\mathbf{d} \;\Rightarrow\; \mathbf{H}(\mathbf{q}_k)\mathbf{d}_k = -\nabla f(\mathbf{q}_k). \tag{8}$$

For $f = \frac{1}{2}\|\mathbf{e}\|^2$, the exact Hessian requires second-order FK terms. The *Gauss–Newton* approximation (3) drops them, giving the regularised normal equations

$$\underbrace{(\mathbf{J}_k^\top \mathbf{J}_k + \lambda \mathbf{I})}_{\mathbf{H}_{\mathrm{GN}} \succ 0} \mathbf{d}_k = -\mathbf{J}_k^\top \mathbf{e}_k, \tag{9}$$

the classic *damped least squares* step. Regularisation bounds $\mathbf{d}_k$ as $\sigma_{\min}(\mathbf{J}) \to 0$.

Theorem 3.1. *Near a non-degenerate solution, GN converges* quadratically*:* $\|\mathbf{q}_{k+1} - \mathbf{q}^*\| \le C\|\mathbf{q}_k - \mathbf{q}^*\|^2$*; digits of accuracy double each iteration. For nonconvex IK this is a local guarantee; a backtracking line search extends convergence from poor starting points.*

The per-iteration cost is $O(n^2)$ for $\mathbf{J}^\top\mathbf{J}$ and $O(n^3)$ for the Cholesky solve. For $n \sim 100$ joints this cubic cost motivates Algorithms 4–5.

Algorithm 3 Gauss–Newton IK Solver

Input: $\mathbf{q}_0$, $\mathbf{g}$, λ, ε
Output: IK solution $\mathbf{q}$

$\mathbf{q} \leftarrow \mathbf{q}_0$
repeat
  $\mathbf{e} \leftarrow \mathrm{FK}(\mathbf{q}) - \mathbf{g}$; **if** $\|\mathbf{e}\| < \varepsilon$ **break**
  $\mathbf{J} \leftarrow \mathbf{J}(\mathbf{q})$; $\mathbf{H} \leftarrow \mathbf{J}^\top\mathbf{J} + \lambda\mathbf{I}$; $\mathbf{g}_\nabla \leftarrow \mathbf{J}^\top\mathbf{e}$
  Solve $\mathbf{H}\mathbf{d} = -\mathbf{g}_\nabla$ ▷ Cholesky, $O(n^3)$
  $\alpha \leftarrow \textsc{Backtrack}(f, \mathbf{q}, \mathbf{d}, \mathbf{g}_\nabla)$; $\mathbf{q} \leftarrow \mathbf{q} + \alpha\mathbf{d}$
until max iterations

4 Algorithm 4 — Quasi-Newton (BFGS)

BFGS avoids forming or inverting $\mathbf{H}$ by maintaining an approximation $\mathbf{B}_k^{-1} \approx \mathbf{H}(\mathbf{q}_k)^{-1}$ updated from gradient differences after every step. Define

$$\mathbf{s}_k = \mathbf{q}_{k+1} - \mathbf{q}_k, \qquad \mathbf{y}_k = \nabla f(\mathbf{q}_{k+1}) - \nabla f(\mathbf{q}_k). \tag{10}$$

The *secant condition* $\mathbf{B}_{k+1}\mathbf{s}_k = \mathbf{y}_k$ determines the unique minimum-change rank-2 update to $\mathbf{B}_k^{-1}$:

$$\mathbf{B}_{k+1}^{-1} = (\mathbf{I} - \rho_k \mathbf{s}_k \mathbf{y}_k^\top)\, \mathbf{B}_k^{-1}\, (\mathbf{I} - \rho_k \mathbf{y}_k \mathbf{s}_k^\top) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top, \quad \rho_k = \tfrac{1}{\mathbf{y}_k^\top \mathbf{s}_k}. \tag{11}$$

The search direction $\mathbf{d}_k = -\mathbf{B}_k^{-1}\mathbf{g}_\nabla$ requires only a matrix-vector product—no linear solve. The update (11) is applied only when $\mathbf{y}_k^\top \mathbf{s}_k > 0$ (positive curvature); otherwise $\mathbf{B}_{k+1}^{-1} = \mathbf{B}_k^{-1}$. Starting from $\mathbf{B}_0^{-1} = \mathbf{I}$, the first step equals GD; curvature information accumulates and the method accelerates.

Theorem 4.1. *BFGS with Wolfe line search converges* superlinearly*:* $\|\mathbf{q}_{k+1} - \mathbf{q}^*\| / \|\mathbf{q}_k - \mathbf{q}^*\| \to 0$*, at $O(n^2)$ cost per iteration versus $O(n^3)$ for Newton. IK is nonconvex, but BFGS adapts its Hessian approximation to local curvature and achieves superlinear-like convergence in practice.*

Algorithm 4 BFGS IK Solver

Input: $\mathbf{q}_0$, $\mathbf{g}$, ε
Output: IK solution $\mathbf{q}$

$\mathbf{q} \leftarrow \mathbf{q}_0$; $\mathbf{B}^{-1} \leftarrow \mathbf{I}_n$; $\mathbf{g}_\nabla \leftarrow \mathbf{J}(\mathbf{q})^\top \mathbf{e}(\mathbf{q})$
repeat
  if $\|\mathbf{e}(\mathbf{q})\| < \varepsilon$ **break**
  $\mathbf{d} \leftarrow -\mathbf{B}^{-1}\mathbf{g}_\nabla$; $\alpha \leftarrow \textsc{Backtrack}(f, \mathbf{q}, \mathbf{d}, \mathbf{g}_\nabla)$
  $\mathbf{q}_+ \leftarrow \mathbf{q} + \alpha\mathbf{d}$; $\mathbf{g}_+ \leftarrow \mathbf{J}(\mathbf{q}_+)^\top \mathbf{e}(\mathbf{q}_+)$
  $\mathbf{s} \leftarrow \mathbf{q}_+ - \mathbf{q}$; $\mathbf{y} \leftarrow \mathbf{g}_+ - \mathbf{g}_\nabla$
  if $\mathbf{y}^\top\mathbf{s} > 10^{-10}$ **then**
    $\rho \leftarrow 1/(\mathbf{y}^\top\mathbf{s})$; $\mathbf{A} \leftarrow \mathbf{I} - \rho\,\mathbf{s}\mathbf{y}^\top$
    $\mathbf{B}^{-1} \leftarrow \mathbf{A}\,\mathbf{B}^{-1}\mathbf{A}^\top + \rho\,\mathbf{s}\mathbf{s}^\top$ ▷ rank-2 update (11)
  end if
  $\mathbf{q} \leftarrow \mathbf{q}_+$; $\mathbf{g}_\nabla \leftarrow \mathbf{g}_+$
until max iterations

5 Algorithm 5 — Nonlinear Conjugate Gradient

BFGS stores $\mathbf{B}^{-1} \in \mathbb{R}^{n\times n}$: $O(n^2)$ memory. For arms with $n \sim 10^2$–10^3 joints this becomes prohibitive. Conjugate Gradient (CG) achieves near-Newton convergence using only $O(n)$ memory by blending the current gradient with the previous search direction:

$$\mathbf{d}_{k+1} = -\nabla f(\mathbf{q}_{k+1}) + \beta_k\,\mathbf{d}_k. \tag{12}$$

The scalar β_k is chosen so successive directions are *conjugate* (curvature-orthogonal) with respect to the Hessian. We use the Polak–Ribière formula with a descent-guarantee clip:

$$\beta_k^{\mathrm{PR}} = \frac{\nabla f(\mathbf{q}_{k+1})^\top[\nabla f(\mathbf{q}_{k+1}) - \nabla f(\mathbf{q}_k)]}{\|\nabla f(\mathbf{q}_k)\|^2}, \qquad \beta_k = \max(0, \beta_k^{\mathrm{PR}}). \tag{13}$$

Conjugacy degrades over time, so the direction resets to steepest descent ($\beta_k = 0$) every n steps to prevent accumulated errors:

$$\beta_k = 0 \quad \text{if } (k+1) \bmod n = 0. \tag{14}$$

Theorem 5.1. *For a strongly convex quadratic, CG with exact line search terminates in $\le n$ steps. For general smooth f, convergence is superlinear with $O(n)$ storage. For nonconvex IK, finite termination does not hold; CG with periodic restart converges to a local minimum with $O(n)$ memory.*

CG and BFGS are complementary: BFGS converges faster but needs $O(n^2)$ storage; CG scales to massive joint spaces. L-BFGS (limited-memory BFGS storing the last $m \ll n$ correction pairs) achieves $O(mn)$ storage with near-BFGS convergence.

Algorithm 5 Nonlinear CG IK Solver (Polak–Ribière + periodic restart)

Input: $\mathbf{q}_0$, $\mathbf{g}$, ε
Output: IK solution $\mathbf{q}$

$\mathbf{q} \leftarrow \mathbf{q}_0$; $\mathbf{g}_\nabla \leftarrow \mathbf{J}(\mathbf{q})^\top \mathbf{e}(\mathbf{q})$; $\mathbf{d} \leftarrow -\mathbf{g}_\nabla$
for $k = 0, 1, 2, \ldots$ **do**
  if $\|\mathbf{e}(\mathbf{q})\| < \varepsilon$ **break**
  $\alpha \leftarrow \textsc{Backtrack}(f, \mathbf{q}, \mathbf{d}, \mathbf{g}_\nabla)$
  $\mathbf{q}_+ \leftarrow \mathbf{q} + \alpha\mathbf{d}$; $\mathbf{g}_+ \leftarrow \mathbf{J}(\mathbf{q}_+)^\top \mathbf{e}(\mathbf{q}_+)$
  $\beta \leftarrow \max\!\big(0,\, \mathbf{g}_+^\top(\mathbf{g}_+ - \mathbf{g}_\nabla)/\|\mathbf{g}_\nabla\|^2\big)$ ▷ Polak–Ribière (13)
  if $(k+1) \bmod n = 0$ **then** $\beta \leftarrow 0$ ▷ periodic restart (14)
  end if
  $\mathbf{d} \leftarrow -\mathbf{g}_+ + \beta\,\mathbf{d}$; $\mathbf{q} \leftarrow \mathbf{q}_+$; $\mathbf{g}_\nabla \leftarrow \mathbf{g}_+$
end for

Comparison Unconstrained Methods

Cost-Function Landscape (2-DOF)

A 2-DOF arm has exactly two IK solutions — *elbow-up* and *elbow-down* — giving exactly **two global minima** at $f = 0$. **GD** follows a long smooth path due to its tiny fixed α; **SD** and **CG** take fewer but fluctuating steps via backtracking; **Newton** and **BFGS** exploit curvature and converge in under ten iterations.

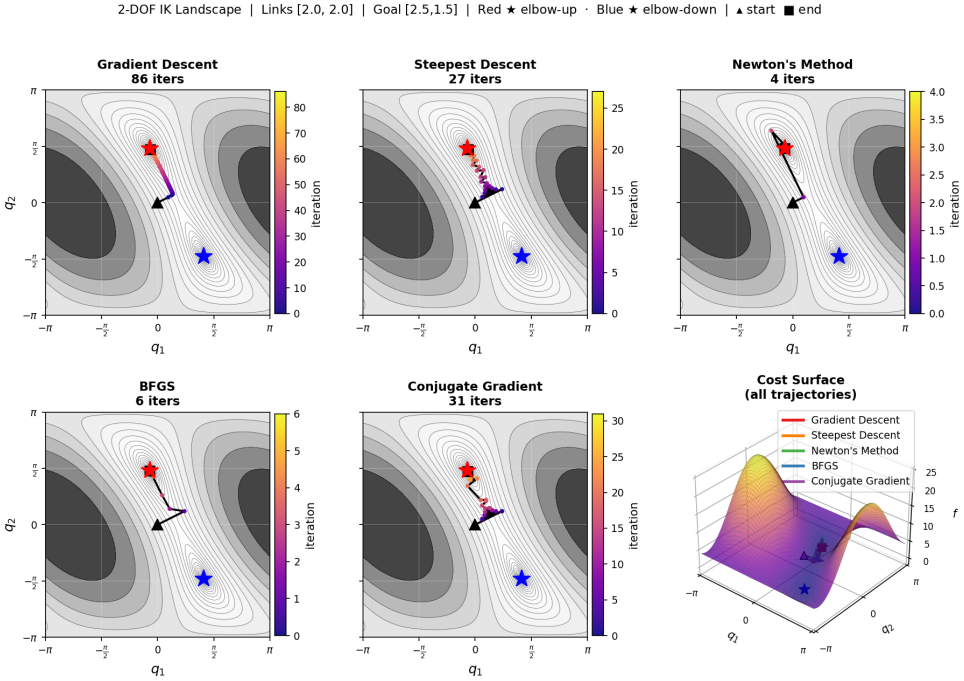

Figure 1: 2-DOF cost landscape with each algorithm's trajectory. Red/blue ★ are the two global minima (elbow-up / elbow-down).

Benchmark Results (5-DOF and 10-DOF, 100 random goals)

Having established qualitative behaviour on a 2-DOF arm, we stress-test all five methods over 100 random goal positions sampled uniformly within the full workspace (Fig. 2), repeated for both 5-DOF and 10-DOF arms.

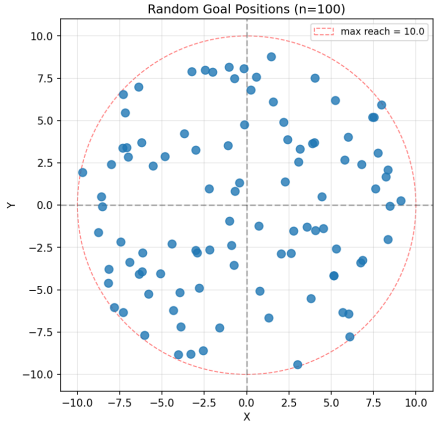

Figure 2: Goal distribution: 100 random targets sampled uniformly within the workspace.

Every method starts from $\mathbf{q}_0 = \mathbf{0}$ and is evaluated on six metrics: convergence rate, average iterations, average end-effector error, wall-clock time, compute speed (iter/ms), and peak parameter storage. All methods achieve **100% convergence** at both DOF settings, and the ranking is consistent across both — the same trends hold whether $n = 5$ or $n = 10$. Tables 1–2 and Figs. 3–4 present the results; best values are **bold**.

Table 1: Performance comparison — 5-DOF arm, 100 goals.

#	Method	Conv%	Avg Iter	Avg Err	Avg Time (ms)	Iter/ms	Params
1	Gradient Descent	**100**	1018.3	0.0997	124.25	**8.20**	**11**
2	Steepest Descent	**100**	67.0	0.0860	20.83	3.22	16
3	Newton's Method	**100**	**6.6**	**0.0253**	**1.53**	4.32	41
4	BFGS	**100**	8.2	0.0478	1.65	4.99	51
5	Conjugate Gradient	**100**	24.1	0.0696	12.22	1.97	22

Table 2: Performance comparison — 10-DOF arm, 100 goals.

#	Method	Conv%	Avg Iter	Avg Err	Avg Time (ms)	Iter/ms	Params
1	Gradient Descent	**100**	306.5	0.0984	104.13	**2.94**	**21**
2	Steepest Descent	**100**	258.8	0.0916	211.51	1.22	31
3	Newton's Method	**100**	**7.2**	**0.0282**	**4.54**	1.58	131
4	BFGS	**100**	11.3	0.0411	5.63	2.00	151
5	Conjugate Gradient	**100**	47.3	0.0751	62.46	0.76	42

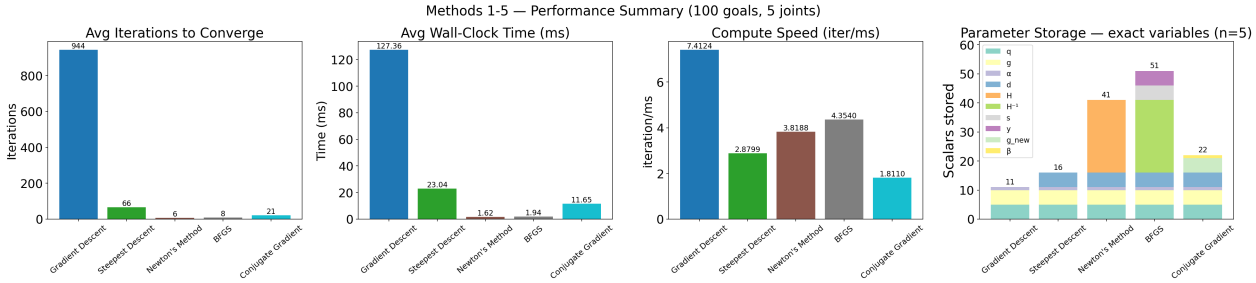

Figure 3: Performance bar charts (5-DOF).

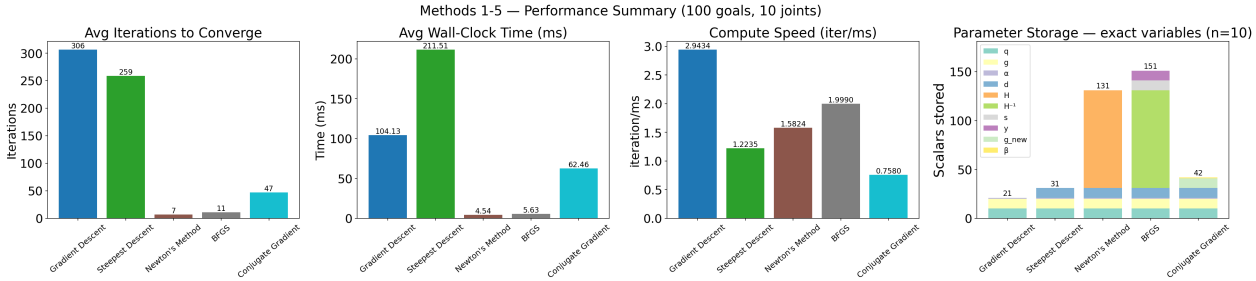

Figure 4: Performance bar charts (10-DOF)

Gradient Descent is the lightest in memory (11 scalars at 5-DOF, 21 at 10-DOF) and has the fastest raw throughput per iteration, but it pays for its tiny fixed step size with an enormous iteration count — over 1 000 at 5-DOF — making it the slowest method in wall-clock time by a wide margin.

Steepest Descent adds Armijo backtracking, which cuts iterations dramatically at 5-DOF (67 vs. 1 018) but at the cost of multiple objective evaluations per step. At 10-DOF the backtracking overhead grows and total time triples, showing that adaptive step selection alone is not enough when the landscape curvature becomes more complex.

Newton's Method is the clear winner on every accuracy and speed metric at both DOF settings, converging in under 8 iterations and under 5 ms regardless of dimension. The trade-off is quadratic storage: the Hessian grows from 41 to 131 scalars as n doubles.

BFGS matches Newton in speed (1.65 ms vs. 1.53 ms at 5-DOF; 5.63 ms vs. 4.54 ms at 10-DOF) by accumulating curvature from gradient differences, without ever forming the Hessian explicitly. Its storage is comparable to Newton ($O(n^2)$) but avoids the $O(n^3)$ linear solve.

Conjugate Gradient is the memory-efficient middle ground: its storage stays $O(n)$ — just 22 scalars at 5-DOF, 42 at 10-DOF — while converging in far fewer iterations than GD by reusing the previous conjugate direction. Per-iteration speed is the lowest of the five due to the Polak-Ribière beta update and backtracking, but the overall iteration count remains moderate.

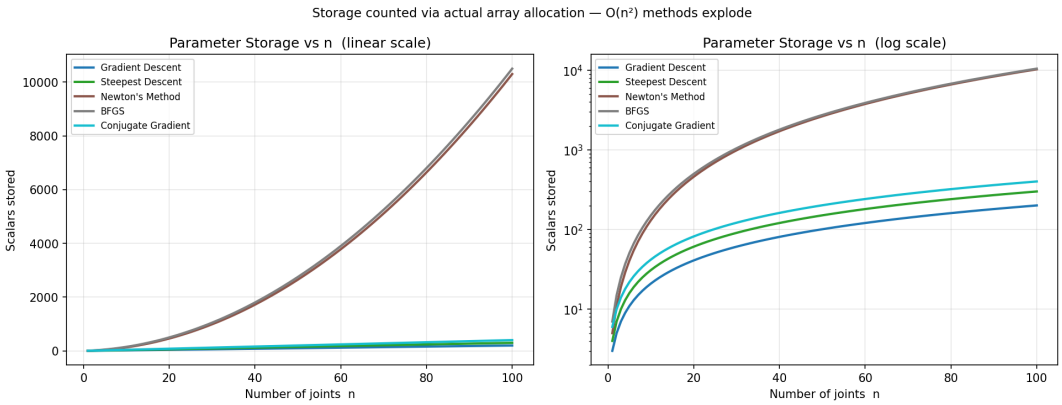

Figure 5: Parameter storage breakdown per method — stacked by variable type (scalars stored at peak memory).

The parameter storage plot (Fig. 5) captures the scaling law: Newton and BFGS grow as $O(n^2)$ while GD, Steepest Descent, and CG grow as $O(n)$, making the latter group increasingly preferable for very-high-DOF arms.

Key Takeaway I: How to Choose an Algorithm

The experiments suggest a simple decision rule based on the available resources and problem size:

- **Use Newton's Method** when n is small-to-moderate and speed is the top priority. It converges in the fewest iterations and lowest wall-clock time across all tested settings, with storage cost $O(n^2)$ that is acceptable for arms with up to tens of joints.
- **Use BFGS** when you want near-Newton performance without ever forming the Hessian. Its accumulated curvature approximation matches Newton's speed while being more robust to poor Hessian conditioning, at the same asymptotic storage class.
- **Use Conjugate Gradient** when memory is limited or n is large. It requires only $O(n)$ storage — identical to plain gradient methods — yet converges in far fewer iterations by reusing the previous conjugate direction. The method of choice for very-high-DOF manipulators.
- **Use Steepest Descent** when the step-size selection matters but storage must stay minimal. Armijo backtracking removes the need to hand-tune α, giving reliable convergence at near-GD memory cost. Avoid it when the landscape has highly elongated level sets.
- **Use Gradient Descent** only as a last resort or a reference baseline. Its fixed step size requires careful hand-tuning and leads to orders-of-magnitude more iterations than any adaptive method, though it is the lightest in storage.

Key Takeaway II: Clear Progression of Methods

#	Method	Search direction $\mathbf{d}_k$	Convergence	Cost/iter	Memory
1	Gradient Descent	$-\nabla f$	$O(1/k)$ linear	$O(n^2)$	$O(n)$
2	Steepest Descent	$-\nabla f$	$O(1/k)$ linear	$O(n^2)$	$O(n)$
3	Newton (GN)	$-\mathbf{H}_k^{-1}\nabla f$	$O(c^{2^k})$ quadratic	$O(n^3)$	$O(n^2)$
4	BFGS	$-\mathbf{B}_k^{-1}\nabla f$	Superlinear	$O(n^2)$	$O(n^2)$
5	Conj. Gradient	$-\nabla f + \beta_k \mathbf{d}_{k-1}$	$\leq n$ steps (quadratic); superlinear (general)	$O(n^2)$	$O(n)$

The five methods form a clear progression, each addressing the key limitation of its predecessor: **GD** (1) is the baseline: one gradient evaluation per step, linear convergence rate, but the fixed α must be hand-tuned and is brittle. **Steepest Descent** (2) keeps the same descent direction but selects α adaptively via Armijo backtracking, eliminating parameter tuning at negligible extra cost. **Newton/GN** (3) replaces the gradient direction with a curvature-scaled Newton step by forming the Hessian explicitly, achieving quadratic convergence at the price of an $O(n^3)$ linear solve per iteration. **BFGS** (4) avoids forming the Hessian by accumulating curvature information incrementally through rank-2 updates (11), matching Newton's convergence rate at $O(n^2)$ cost. **CG** (5) recovers $O(n)$ memory by building conjugate search directions (12) from only the previous gradient and step, making it the method of choice for very-high-DOF manipulators.

Part II — Equality Constrained Methods (Algorithms 6–7)

Algorithms 1–5 minimise $f(\mathbf{q})$ freely over $\mathbb{R}^n$. Part II adds hard constraints. **Algorithms 6–7** enforce joint limits and an equality constraint requiring the first two joints to lie on the unit circle:

$$\begin{aligned} \min_{\mathbf{q}} \; & f(\mathbf{q}) = \tfrac{1}{2}\|\mathrm{FK}(\mathbf{q}) - \mathbf{g}\|^2, \\ \text{s.t.} \; & \sum_{i=1}^{n} q_i^n = 1, \\ & q_{\min,i} \le q_i \le q_{\max,i}, \quad \forall\, i, \end{aligned} \tag{15}$$

with $q_{\min,i} = -\pi/2$, $q_{\max,i} = \pi/2$ and experiments using a 2-DOF arm ($n = 2$, $\ell_1 = \ell_2 = 2$).

Both Newton-KKT (Alg. 6) and Infeasible-Start Newton (Alg. 7) are derived for problems with *equality* constraints $h(\mathbf{q}) = \mathbf{0}$. Joint limits are *inequality* constraints, so an active-set strategy is used to bridge the gap: active bounds are promoted to equalities each iteration, and dropped when they are no longer binding.

Active-Set Strategy for Box Constraints (shared by Algs. 6–7)

Input: $\mathbf{q}$, $\mathbf{q}_{\min}, \mathbf{q}_{\max}$, $\varepsilon_{\mathcal{A}}$, $\boldsymbol{\lambda}$
Output: $\mathcal{A}$, $\mathcal{F}$

1: $\mathcal{A} \leftarrow \{i : q_i \le q_{\min,i} + \varepsilon_{\mathcal{A}}\} \cup \{i : q_i \ge q_{\max,i} - \varepsilon_{\mathcal{A}}\}$; $\mathcal{F} \leftarrow \{1, \dots, n\} \setminus \mathcal{A}$
2: Solve constrained Newton over $\mathcal{F} \Rightarrow \Delta\mathbf{q}_{\mathcal{F}}, \boldsymbol{\lambda}$; $\Delta\mathbf{q}_i \leftarrow 0 \; \forall i \in \mathcal{A}$
3: $\mathbf{q} \leftarrow \mathrm{clip}(\mathbf{q} + \alpha\, \Delta\mathbf{q},\, \mathbf{q}_{\min}, \mathbf{q}_{\max})$
4: $\mathcal{A} \leftarrow \mathcal{A} \setminus \{i \in \mathcal{A} : \lambda_i < 0\}$

6 Algorithm 6 — Newton with KKT (Active-Set + Equality)

At each iterate $\mathbf{q}_k$, the *active set* $\mathcal{A} = \{i : |q_i - q_{\min,i}| < \varepsilon_{\mathcal{A}}\} \cup \{i : |q_i - q_{\max,i}| < \varepsilon_{\mathcal{A}}\}$ collects joints at a bound. Active constraints are promoted to equalities, and together with the equality constraint $h(\mathbf{q}) = 0$ the Newton step solves the $(n + 1 + |\mathcal{A}|) \times (n + 1 + |\mathcal{A}|)$ KKT system:

$$\begin{bmatrix} \mathbf{H} + \nu\nabla^2 h & \mathbf{c} & \mathbf{A}^\top \\ \mathbf{c}^\top & 0 & \mathbf{0} \\ \mathbf{A} & \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \Delta\mathbf{q} \\ \Delta\nu \\ \Delta\boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} -\mathbf{g}_{\mathrm{res}} \\ -h(\mathbf{q}) \\ \mathbf{0} \end{bmatrix}, \tag{16}$$

where $\mathbf{c} = \nabla h = [2q_1, 2q_2, 0, \dots]^\top$, $\nabla^2 h = \mathrm{diag}(2, 2, 0, \dots)$, and ν is the equality multiplier. Any active constraint with $\lambda_i < 0$ is dropped; an L1-penalty merit $f + \mu|h|$ drives Armijo backtracking. The initial point is projected onto the unit circle then clipped to the box, so feasibility of (15) holds from step 1.

Algorithm 6 Newton+KKT+Equality IK (Algorithm 6)

Input: $\mathbf{q}_0$, $\mathbf{g}$, joint limits, ε
Output: $\mathbf{q}$, ν

1: $\mathbf{q} \leftarrow \mathrm{clip}\big(\frac{(q_1,q_2)}{\|(q_1,q_2)\|} \oplus \mathbf{q}_{3:n}\big)$; $\nu \leftarrow 0$
2: **repeat**
3: **if** $\|\mathrm{FK}(\mathbf{q}) - \mathbf{g}\| < \varepsilon$ and $|h(\mathbf{q})| < \varepsilon$ **break**
4: $\mathcal{A} \leftarrow \textsc{ActiveSet}(\mathbf{q})$; build $\mathbf{H}, \mathbf{g}_\nabla, \mathbf{c}, h$
5: **repeat** Solve (16); drop $\lambda_i < -10^{-8}$ from $\mathcal{A}$
6: **until** no negative multipliers
7: $\mathbf{q} \leftarrow \mathrm{clip}(\mathbf{q} + \textsc{Armijo}(f + \mu|h|) \cdot \Delta\mathbf{q})$; $\nu \leftarrow \nu + \Delta\nu$
8: **until** max iterations

7 Algorithm 7 — Infeasible-Start Newton

The *infeasible Newton* method drives all residuals to zero simultaneously without requiring a feasible start. With dual variables $\boldsymbol{\lambda}_{\text{lo}}, \boldsymbol{\lambda}_{\text{hi}} \geq \mathbf{0}$, slacks $\mathbf{s}_{\text{lo}} = \mathbf{q} - \mathbf{q}_{\min}$, $\mathbf{s}_{\text{hi}} = \mathbf{q}_{\max} - \mathbf{q}$, and equality multiplier ν, the perturbed KKT conditions are

$$\nabla f + \nu \nabla h - \boldsymbol{\lambda}_{\text{lo}} + \boldsymbol{\lambda}_{\text{hi}} = \mathbf{0}, \quad h(\mathbf{q}) = 0, \quad \boldsymbol{\Lambda}_{\text{lo}}\mathbf{s}_{\text{lo}} = \tau\mathbf{1}, \quad \boldsymbol{\Lambda}_{\text{hi}}\mathbf{s}_{\text{hi}} = \tau\mathbf{1}. \tag{17}$$

Eliminating $\Delta\boldsymbol{\lambda}_{\text{lo}}, \Delta\boldsymbol{\lambda}_{\text{hi}}$ yields the condensed $(n+1) \times (n+1)$ saddle-point system:

$$\begin{bmatrix} \mathbf{H} + \nu\nabla^2 h + \mathbf{D}_{\text{lo}} + \mathbf{D}_{\text{hi}} & \mathbf{c} \\ \mathbf{c}^\top & 0 \end{bmatrix} \begin{bmatrix} \Delta\mathbf{q} \\ \Delta\nu \end{bmatrix} = \begin{bmatrix} -\mathbf{b}_{\mathbf{q}} \\ -h(\mathbf{q}) \end{bmatrix}, \tag{18}$$

where $\mathbf{D}_\bullet = \text{diag}(\lambda_{\bullet,i}/s_{\bullet,i})$ and $\mathbf{b}_{\mathbf{q}}$ is the stationarity residual including $\nu\nabla h$. The barrier terms $\mathbf{D}_{\text{lo}} + \mathbf{D}_{\text{hi}}$ diverge as $s \to 0$, preventing constraint violation. Dual steps $\Delta\boldsymbol{\lambda}$ are recovered analytically; step sizes follow the *fraction-to-boundary rule* (0.995). $\nu_0 = 0$; τ is reduced by factor 0.1 after each inner loop until $\tau < 10^{-10}$.

Algorithm 7 Infeasible Newton+Equality IK (Algorithm 7)

Input: $\mathbf{q}_0$, $\mathbf{g}$, $\tau_0 = 1$, ε
Output: $\mathbf{q}$, ν

$\mathbf{q} \leftarrow \text{clip}(\mathbf{q}_0, \mathbf{q}_{\min}+\epsilon, \mathbf{q}_{\max}-\epsilon)$; $\boldsymbol{\lambda} \leftarrow \tau_0/\mathbf{s}$; $\nu \leftarrow 0$
while $\tau > \tau_{\min}$ **do**
  repeat ▷ inner Newton loop
    if $\|\text{FK}(\mathbf{q}) - \mathbf{g}\| < \varepsilon$ and $|h(\mathbf{q})| < \varepsilon$ and $\tau < 10^{-6}$ **break**
    Solve (18) $\Rightarrow \Delta\mathbf{q}, \Delta\nu$; recover $\Delta\boldsymbol{\lambda}$
    $\mathbf{q} \leftarrow \text{clip}(\mathbf{q} + \alpha_p\Delta\mathbf{q})$; $\boldsymbol{\lambda} \leftarrow \boldsymbol{\lambda} + \alpha_d\Delta\boldsymbol{\lambda}$; $\nu \leftarrow \nu + \Delta\nu$
  until inner max iterations
  $\tau \leftarrow \max(\tau \times 0.1,\ \tau_{\min})$
end while

Comparison Equality Constrained Methods

We test both methods over 50 random initial configurations drawn from three physically valid regions (all inside the joint-limit box, since a real robot arm cannot start outside its own mechanical limits):

- **Inside box**: $\mathbf{q}_0$ uniform in $[-\pi/2, \pi/2]^2$, *outside* the unit circle ($\|\mathbf{q}_0\| > 1$).
- **Inside circle**: $\mathbf{q}_0$ uniform in the box *and* inside the unit circle ($\|\mathbf{q}_0\| < 1$).
- **On arc**: $\mathbf{q}_0$ sampled directly from the arc that lies within the box (already equality-feasible).

Table 3: Success-rate benchmark, Algs. 6 vs. 7 (2-DOF, 50 trials/region). MedIter: median iterations for converged trials. Mean$|h|$: residual equality violation.

Region	Method	Success%	MedIter	MeanDist	Mean$\|h\|$
Inside box	6 Newton+KKT	92	163.8	0.331	0.005
	7 Infeasible	94	1436.9	0.260	0.000
Inside circle	6 Newton+KKT	88	243.8	0.496	0.006
	7 Infeasible	94	1436.9	0.268	0.000
On arc	6 Newton+KKT	94	123.8	0.250	0.009
	7 Infeasible	94	1448	0.259	0.000

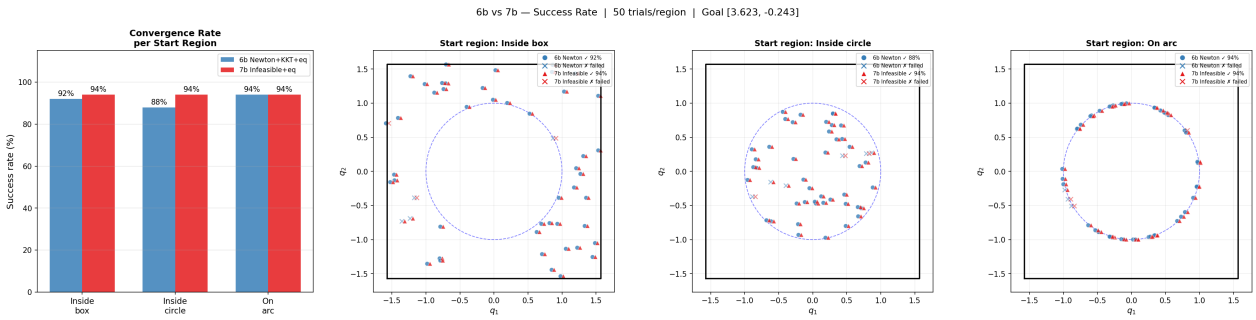

Figure 6: Success-rate bar chart and start-point scatter for each region. Blue circles = Alg. 6 converged; red triangles = Alg. 7 converged; crosses = failed. The dashed circle is the equality constraint manifold; the black square is the joint-limit box.

Alg. 7 achieves higher or equal success from every region because it never requires an equality-feasible start: it treats $h(\mathbf{q}) = 0$ as a residual and drives it to zero jointly with the IK objective, navigating freely through the infeasible interior. Alg. 6 must project $\mathbf{q}_0$ onto the arc first; a Euclidean projection is not guaranteed to land near the cost minimum on the arc, so bad projections stall the KKT system and lower success to 88–92 % off-arc. When the start is already on the arc (*On arc* region), no projection is needed and both methods tie at 94 %. However, when Alg. 6 *does* converge it is ≈10× faster (124–244 vs. 1 437 iterations), because each KKT step directly solves the constrained Newton system without a barrier schedule. The trade-off is constraint accuracy: the barrier schedule of Alg. 7 enforces $h = 0$ to machine precision ($\texttt{Mean}|h| = 0$), while simultaneous active box constraints can make the Alg. 6 KKT system ill-conditioned, leaving a small residual ($\texttt{Mean}|h| \approx 0.007$).

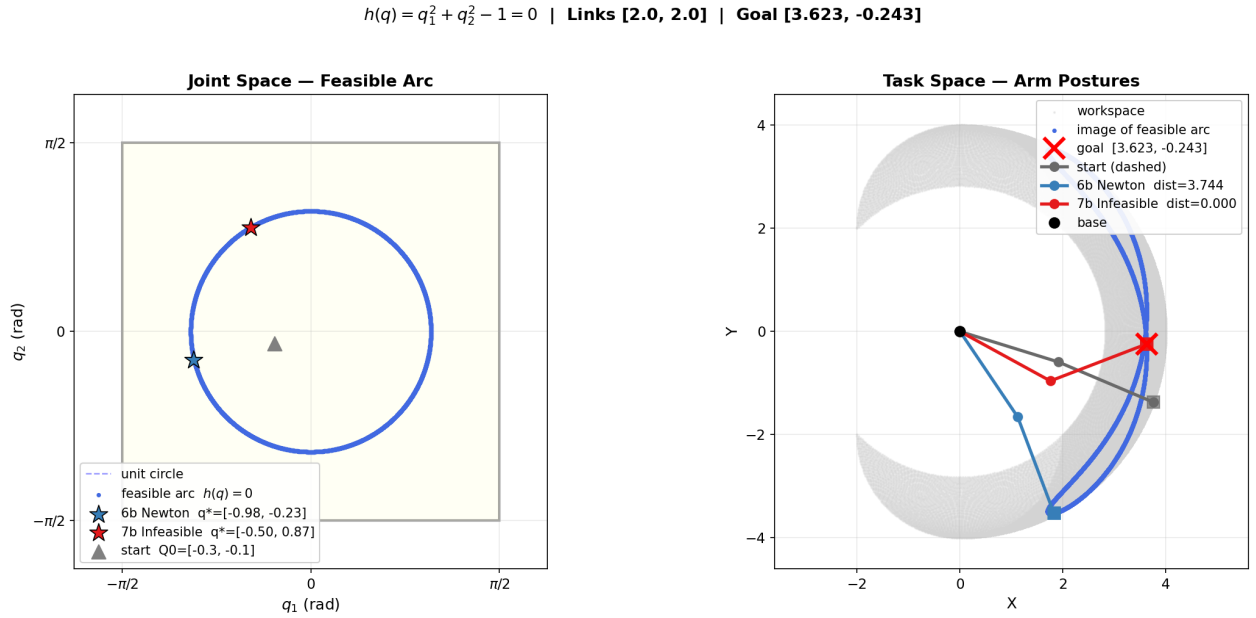

Figure 7: Feasible set $h(\mathbf{q}) = q_1^2 + q_2^2 - 1 = 0$ ($\ell_1 = \ell_2 = 2$, goal $[3.623, -0.243]$). **Left**: joint space — unit-circle arc inside $[-\pi/2, \pi/2]^2$. **Right**: task space — start (dashed) and converged arm postures.

Fig. 7–8 illustrate convergence behaviour under a poor initial guess. Alg. 6 clamps $\mathbf{q}_0$ to the nearest arc point and then solves the KKT system constrained to the arc; if that point lies in a flat region of $f|_{\text{arc}}$, the Newton step is near-zero and the solver is trapped. Alg. 7 does not clamp: because h enters the primal residual via the dual variable ν, the iterates are free to cross the infeasible region and approach the arc from any direction, reaching a lower-cost point on the manifold.

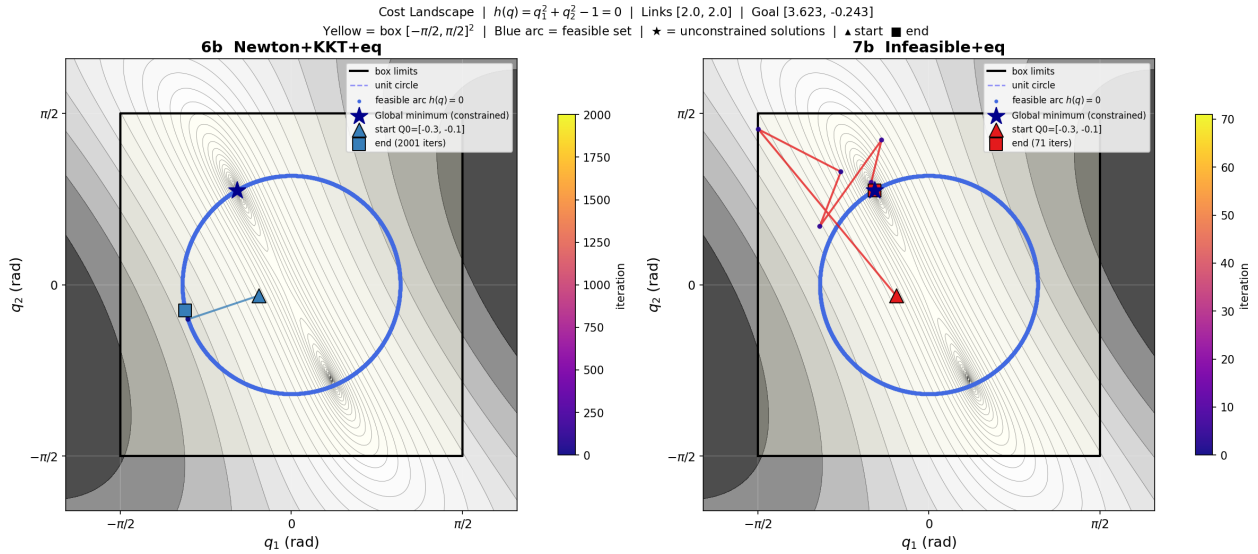

Figure 8: Cost landscape $f(q_1, q_2)$; blue arc = feasible set $h(\mathbf{q}) = 0$; ★ = unconstrained solutions; colour = iteration count. **Left**: Alg. 6 projects onto the arc then converges in few steps. **Right**: Alg. 7 approaches the arc from the interior with more iterations.

Conclusion. Alg. 7 (Infeasible Newton) is the safer default: it succeeds from any interior start and satisfies $h = 0$ exactly, at the cost of $\approx 10\times$ more iterations. Alg. 6 (Newton+KKT) is preferred only with a warm start on or near the arc, where its direct KKT solve is faster and the bad-projection risk is eliminated.

Part III — Unequality Constrained Method (Joint Limits + Obstacle Avoidance)

---

8 Algorithm 8 — Interior Point with Log-Barrier

Algorithm 8 extends (15) by additionally requiring every joint position $\mathbf{p}_k(\mathbf{q})$ to clear an obstacle of centre $\mathbf{c}$ and radius r by margin δ:

$$\begin{aligned} &\min_{\mathbf{q}} \; f(\mathbf{q}) = \tfrac{1}{2}\|\mathrm{FK}(\mathbf{q}) - \mathbf{g}\|^2. \\ &\text{s.t.} \quad q_{\min,i} \le q_i \le q_{\max,i}, \; \|\mathbf{p}_k(\mathbf{q}) - \mathbf{c}\| \ge r + \delta, \quad \forall\, i, k. \end{aligned} \tag{19}$$

Both constraint types are encoded as log-barrier terms in the objective. The augmented cost is

$$F_\mu(\mathbf{q}) = f(\mathbf{q}) + \mu\, \varphi(\mathbf{q}), \tag{20}$$

where the barrier φ penalises proximity to every constraint boundary:

$$\varphi(\mathbf{q}) = -\sum_{i=1}^{n} \big[\log(q_i - q_{\min,i}) + \log(q_{\max,i} - q_i)\big] - \sum_{k=1}^{n} \log\big(\|\mathbf{p}_k(\mathbf{q}) - \mathbf{c}\| - r - \delta\big). \tag{21}$$

As $\mu \to 0$, $F_\mu \to f$ and the solution of $\min F_\mu$ converges to the exact constrained optimum along the *central path*. At each outer iteration (fixed μ), a Newton step is taken on (20):

$$\mathbf{H}_\mu\, \Delta\mathbf{q} = -\mathbf{g}_\mu, \qquad \mathbf{g}_\mu = \mathbf{g}_f + \mu(\mathbf{g}_j + \mathbf{g}_o), \quad \mathbf{H}_\mu = \mathbf{H}_f + \mu\, \mathrm{diag}(\mathbf{h}_j + \mathbf{h}_o), \tag{22}$$

where subscripts f, j, o denote contributions from the objective, joint barrier, and obstacle barrier respectively. The diagonal structure of the barrier Hessian keeps the cost at $O(n^3)$ (same as Newton) with no matrix outer products. μ is reduced by factor 0.1 after each inner loop, starting from $\mu_0 = 1$.

Algorithm 8 Log-Barrier Interior-Point IK

Input: $\mathbf{q}_0$, $\mathbf{g}$, $\mu_0 = 1$, scale $= 0.1$, ε
Output: Constrained IK solution $\mathbf{q}$

$\mathbf{q} \leftarrow \text{clip}(\mathbf{q}_0, \mathbf{q}_{\min} + \epsilon, \mathbf{q}_{\max} - \epsilon)$ ▷ start strictly feasible
while $\mu > \mu_{\min}$ **do**
  repeat ▷ inner Newton loop on F_μ
    Compute $\mathbf{g}_\mu, \mathbf{H}_\mu$ from (22)
    Solve $\mathbf{H}_\mu \Delta\mathbf{q} = -\mathbf{g}_\mu$
    $\alpha \leftarrow 1$; halve α until φ finite and $F_\mu(\mathbf{q} + \alpha\Delta\mathbf{q}) \le F_\mu(\mathbf{q}) + 10^{-4}\alpha\,\mathbf{g}_\mu^\top\Delta\mathbf{q}$
    $\mathbf{q} \leftarrow \text{clip}(\mathbf{q} + \alpha\Delta\mathbf{q})$
    if goal reached and $\mu < 10^{-4}$ **break**
  until inner max iterations
  $\mu \leftarrow \max(\mu \times 0.1,\ \mu_{\min})$
end while

Results — 3-DOF Redundant Arm

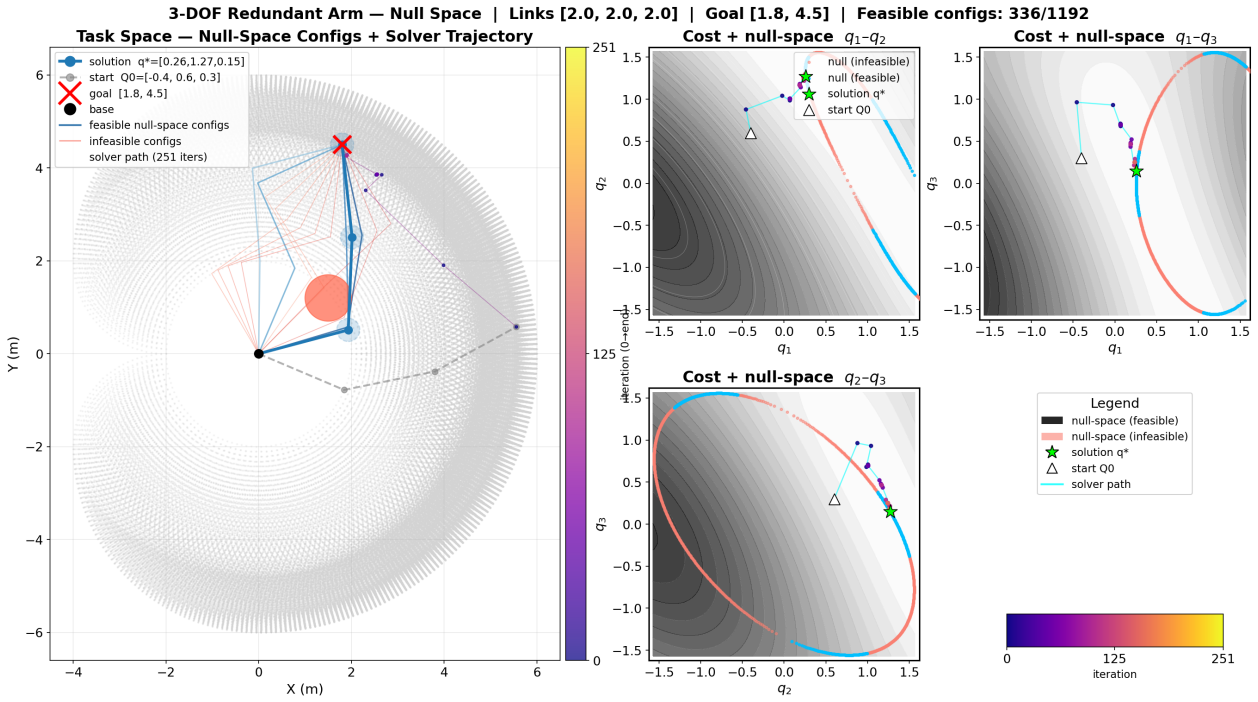

Figure 9: Interior-point IK on a 3-DOF planar arm (ℓ_1=ℓ_2=ℓ_3=2 m, joint limits $[-90, 90]^3$, obstacle at (1.5, 1.2) with r=0.5, clearance δ=0.25). **Left**: task space showing sampled null-space arm configurations (blue = feasible, red = infeasible) and the solver end-effector trajectory (plasma colourmap, dark → light = early → late iterations). Dashed blue circles at each joint of the solution arm show the clearance radius δ. **Right panels**: cost landscape $f(q_i, q_j)$ (grey, fixed third joint at q^*) with the null-space curve and plasma-coloured solver path projected onto each joint pair.

Algorithm 8 requires **no active-set bookkeeping**: the log-barrier encodes *all* inequality constraints — joint limits and obstacle clearance — simultaneously into the objective, so no constraint is ever explicitly activated or dropped. The iterate is always strictly feasible, and only the penalty weight μ needs to decrease over time. Fig. 9 confirms this on a redundant 3-DOF arm: the

solver trajectory (plasma path) descends through the cost landscape to $q^* = [-0.65, 1.28, 0.01]$ rad, reaching the goal within 0.05 m and maintaining obstacle clearance > 0.25 m at every joint without ever explicitly identifying which constraints are active.

Limitation — disconnected feasible basins. The goal may be geometrically reachable and within the joint-limit box, yet still inaccessible. When an obstacle divides the feasible region into separate connected components, the log-barrier creates an impenetrable wall around it. The solver is trapped in whichever basin contains the start and cannot navigate around the obstacle to reach the goal in the other basin, even though a valid IK solution exists there. This is a direct consequence of the strictly-interior property: the iterate never crosses a barrier boundary, so topology changes in the feasible set caused by obstacles are invisible to the solver.

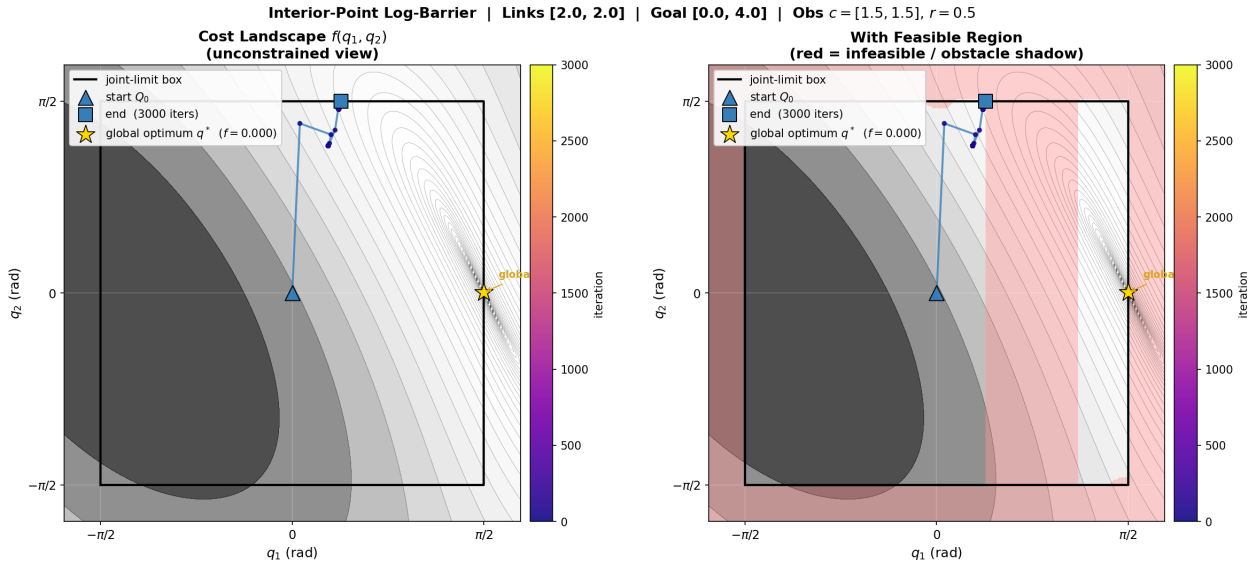

Figure 10: 2-DOF cost landscape: goal is within the joint-limit box but separated from the start by an obstacle. The solver stalls on the wrong side — the barrier wall blocks crossing to the feasible basin containing the goal.

Key Takeaway III: Constrained Optimisation Strategy

#	Method	Constraint type	Convergence	Trade-off
6	Newton + KKT	Inequality → equality	Quadratic	Fast but needs feasible warm start
7	Infeasible Newton	Equality (residual)	≈10× slower	Explores infeasible set; robust from any start
8	Interior-Point	Pure inequality	Moderate (μ schedule)	No bookkeeping; blind to disconnected basins

If the inequality can be reformulated as equality, **Alg. 6** converges fastest. When no feasible warm start exists, **Alg. 7** explores the infeasible interior to find solutions at the cost of more iterations. For pure inequality constraints (joint limits, obstacles), **Alg. 8** is the only choice — but its strictly-interior property makes it blind to goal basins blocked by obstacle walls.

References

[1] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Paques, "Python-Robotics: a Python code collection of robotics algorithms," *arXiv preprint arXiv:1808.10703*, 2018. https://github.com/AtsushiSakai/PythonRobotics/tree/master/ArmNavigation/n_joint_arm_to_point_control

[2] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Pearson Prentice Hall, 2005.

[3] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling and Control*. Wiley, 2006.

[4] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Technical Report TR 98-11, Iowa State University, 1998.

[5] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, Cambridge, UK, 2004.